

URIB Engineering Division | Internal Technical Documentation

Universal Bridge

URIB

Developer Onboarding Guide

Technical Reference — URIB Engineering

Document Date: July 2026

Audience: Backend / Systems Software Engineers — URIB Team

Maintained by: URIB Engineering

Version 1.0 • First Issue

Table of Contents

1	Introduction & Purpose
1.1	What Is URIB?
1.2	Goals & Objectives
1.3	Who Uses URIB?
1.4	How to Use This Guide
2	Conceptual Foundations
2.1	The Rail Abstraction Model
2.2	The Commitment Model
2.3	Invariants (INV-001 through INV-005)
2.4	Math Primitives
3	Architecture Overview
3.1	High-Level Component Map
3.2	Data Flow Diagram (Prose)
3.3	Deployment Topology
4	Module Breakdown
5	Function Interfaces
5.1	IRailAdapter Interface
5.2	CommitmentManager API
5.3	BridgeSwapCoordinator API
5.4	AuditLedger API
5.5	MathPrimitives

1	Introduction & Purpose
6	Rail Mappings
7	Settlement Outputs
7.1	SettlementReceipt
7.2	MerkleProof
7.3	AuditBatch
8	Audit Flows
8.1	Real-Time Event Streaming
8.2	Batch Sealing & Merkle Anchoring
8.3	Dispute Resolution Flow
8.4	Compliance Exports
9	Developer Quickstart
10	Glossary
11	Implementing Additional Rail Adapters
11.1	When to Add a New Rail
11.2	Implementation Checklist
11.3	Mock Adapter Pattern
11.4	Blockchain Adapter Extra Requirements
12	Sequence Diagrams
12.1	Standard Commitment Lifecycle — Happy Path
12.2	Bridge Swap Flow — Cross-Rail
12.3	Dispute Resolution Flow
12.4	New Rail Adapter Registration Flow

1	Introduction & Purpose
13	Error Code Reference
13.1	Error Envelope Schema
13.2	Gateway Errors — URIB-GW
13.3	Commitment Manager Errors — URIB-CM
13.4	Rail Dispatcher Errors — URIB-RD
13.5	Rail Adapter Errors — URIB-RA
13.6	Settlement Engine Errors — URIB-SE
13.7	Bridge Swap Coordinator Errors — URIB-BSC
13.8	Audit Ledger Errors — URIB-AUD
13.9	Invariant Violation Errors — URIB-INV
14	Runbook Appendix
RB-001	Commitment Stuck in LOCKED State
RB-002	Partial Swap Failure (URIB-BSC-002)
RB-003	Blockchain Block Reorg (URIB-RA-015)
RB-004	Invariant Violation CRITICAL (URIB-INV)
RB-005	Rail Adapter Outage

1. Introduction & Purpose

1.1 What Is URIB?

The **Universal Bridge (URIB)** is a universal financial-rail abstraction discipline that bridges heterogeneous settlement networks, payment rails, and liquidity pools under a single, mathematically invariant interface. Rather than exposing the idiosyncratic protocols of individual settlement networks — such as ACH batch windows, SWIFT message formats, or on-chain gas mechanics — URIB presents all downstream consumers with a uniform surface: the **Commitment** primitive. Engineers working above the URIB layer never interact with rail-specific APIs, message schemas, or latency profiles directly. Those concerns live exclusively within Rail Adapters, governed by the Rail Capability Matrix.

URIB is not a payment processor, a ledger, or a reconciliation tool in isolation. It is a *discipline*: a set of composable interfaces, lifecycle contracts, and mathematical guarantees that together ensure correctness, auditability, and extensibility regardless of which underlying settlement rails are in use at any given moment. New rails can be added by implementing a single interface and registering one entry in the Rail Capability Matrix — without touching any application-layer code.

1.2 Goals & Objectives

- **Eliminate rail-specific business logic from application layers.** All rail behavior is encapsulated in Rail Adapters and described declaratively in the Rail Capability Matrix. Application engineers must never branch on rail identifiers in business logic.
- **Guarantee settlement finality proofs.** Every settled Commitment produces a cryptographically verifiable `SettlementReceipt` whose `finality_proof_hash` is anchored to an append-only Audit Ledger and periodically committed to an external notary via Merkle roots.
- **Provide composable primitives for multi-rail transactions.** The Bridge Swap Coordinator chains two Commitments through an internal liquidity pool, enabling atomic cross-rail value transfer without exposing the complexity of two-phase settlement to the calling layer.
- **Enforce mathematical invariants at every boundary.** Five core invariants — Conservation, Idempotency, Finality Monotonicity, Rail Isolation, and Denomination Integrity — are enforced as compile-time constraints and runtime assertions throughout the system.

1.3 Who Uses URIB?

Audience	Interaction with URIB
Internal Financial Platform Engineers	Submit Commitments via the URIB Gateway; consume SettlementReceipts to update product-layer ledgers.
Rail Integration Teams	Implement <code>IRailAdapter</code> for new rails; register rail traits in the Rail Capability Matrix; own adapter deployments.
Audit & Compliance Engineers	Query the Audit Ledger for LifecycleEvents and Merkle proofs; export AuditBatches for regulatory reporting; verify finality proofs in dispute resolution.
Settlement Infrastructure Engineers	Operate and scale the Commitment Store, Event Bus, Settlement Engine, and Bridge Swap Coordinator.

1.4 How to Use This Guide

This document is intended for experienced backend and systems developers who are new to the URIB discipline. It assumes familiarity with the following prerequisite domains:

- Distributed ledger design and double-entry accounting primitives
- Message queue architectures (specifically Kafka topics, consumer groups, and offset management)
- Fixed-point arithmetic and its application in financial systems
- Cryptographic primitives: hash functions (SHA-256, BLAKE3), Merkle trees, and digital signatures
- State machine modeling and lifecycle management in distributed systems

Recommended reading order for new engineers: Read Sections 1 and 2 first to internalize the conceptual model and invariants — these are foundational and must be understood before touching any code. Proceed to Section 3 for the architectural picture, then Section 4 for the module map. Section 5 is the primary technical reference for day-to-day implementation work. Sections 6 through 8 are reference material for rail-specific and compliance scenarios. Section 9 is the hands-on quickstart for getting a local environment running on day one.

⚠ Prerequisite Notice

Engineers who have not worked with fixed-point arithmetic in a financial context are strongly advised to read Section 2.4 (Math Primitives) and the `urib.math`

module documentation before attempting to write any code that handles monetary values. Using floating-point types anywhere in the value path is a critical defect.

2. Conceptual Foundations

2.1 The Rail Abstraction Model

In URIB terminology, a **Rail** is any settlement network — a batch ACH file exchange, a real-time SWIFT message, an RTP credit push, a stablecoin transfer on an EVM-compatible chain, or an instantaneous internal book transfer — abstracted behind a uniform `IRailAdapter` interface. From the perspective of any component above the Rail Dispatcher, all rails are identical: they accept a Commitment, perform settlement, and return a receipt. The specifics of how that settlement occurs are entirely encapsulated within the Rail Adapter.

Each rail is characterized by a fixed set of **Rail Traits**:

Trait	Description	Example Values
Unidirectionality	Whether value flow is one-way only on this rail (most rails are unidirectional; book transfers are bidirectional).	<code>true</code> (ACH, RTP), <code>false</code> (Book)
Finality Latency	The expected time from submission to irrevocable settlement confirmation.	T+1 day, <10 seconds, ~12 seconds
Denomination	The currency or asset class this rail settles in.	USD, USDC, Multi-currency, Internal units
Capacity Limits	Maximum single-transaction value the rail can settle.	\$25,000,000 (ACH), \$1,000,000 (RTP), None (SWIFT)
Idempotency Guarantees	Whether the rail natively supports deduplication via a stable reference key.	Trace number (ACH), UETR (SWIFT), Tx hash (Stablecoin)
Reversibility	Whether a settled transaction can be reversed or recalled within a defined window.	Yes (ACH within window), No (RTP, SWIFT, Stablecoin)

The **Rail Capability Matrix (RCM)** is a compile-time configuration table — defined in `urib.config.rcm` — that maps every known `RailID` to its complete set of Rail Traits. The RCM is the single source of truth for all rail behavior. Developers must *never* hard-code rail behavior assumptions in business logic or adapter code. If a trait needs to be consulted (for example, to determine whether a Commitment amount exceeds a rail's capacity limit), the RCM must be queried programmatically. This ensures that changes to rail characteristics — such as a regulator raising the RTP per-transaction cap — require only a single RCM update, with no code changes required elsewhere.

RCM Rule

All rail trait lookups must go through

`RailCapabilityMatrix.get(rail_id)`

. Never hard-code capacity limits, latency assumptions, or reversibility flags. The RCM is validated at service startup; a malformed or incomplete RCM entry will prevent the service from starting.

2.2 The Commitment Model

A **Commitment** is the central primitive of URIB. It is a cryptographically bound, time-gated promise to settle a specified value transfer on a specific rail. Once issued, a Commitment is immutable: its fields cannot be modified by any actor in the system. The only permissible operations on an existing Commitment are state transitions, which are governed exclusively by the `CommitmentManager` and, for the `LOCKED-to-SETTLED/EXPIRED` transition, the Settlement Engine.

Commitment Fields

Field	Type	Description
<code>commit_id</code>	UUID	Globally unique identifier for this Commitment. Also serves as the idempotency key for INV-002.
<code>rail_id</code>	RailID	Identifies the settlement rail to which this Commitment is exclusively bound (INV-004).
<code>amount</code>	int (fixed-point)	The settlement value as a fixed-point integer. Immutable after issuance (INV-005).
<code>scale</code>	int	The scale factor defining the decimal position. E.g., <code>amount=10050</code> , <code>scale=2</code> represents \$100.50.
<code>currency</code>	Currency	ISO 4217 currency code or asset identifier (e.g., <code>USD</code> , <code>USDC</code>). Immutable after issuance.
<code>expiry_epoch</code>	int (Unix epoch)	The Unix timestamp at which a PENDING or LOCKED Commitment automatically transitions to EXPIRED.
<code>sender_handle</code>	str	Pseudonymous identifier for the value sender. Resolved to a real identity only by authorized compliance systems.
<code>receiver_handle</code>	str	Pseudonymous identifier for the value receiver.
<code>memo_hash</code>	bytes (32)	SHA-256 hash of the plaintext memo string. The original memo is never stored in the Commitment record.

Commitment Lifecycle State Machine

Every Commitment follows a strict, monotonic lifecycle. The only permitted state transitions are shown below. Any attempt to perform a non-permitted transition will raise a `StateMachineViolation` exception and be rejected by the CommitmentManager.

From State	To State	Trigger	Actor
PENDING	LOCKED	<code>lock_commitment()</code> called	CommitmentManager
PENDING	EXPIRED	Current time > <code>expiry_epoch</code>	CommitmentManager (scheduled)
PENDING	REVOKED	<code>revoke_commitment()</code> called	CommitmentManager (authorized callers only)
LOCKED	SETTLED	Rail Adapter returns settlement confirmation	Settlement Engine only
LOCKED	EXPIRED	Settlement not confirmed before <code>expiry_epoch</code>	Settlement Engine only

From State	To State	Trigger	Actor
SETTLED	<i>(none)</i>	Terminal state — INV-003	N/A
EXPIRED	<i>(none)</i>	Terminal state — INV-003	N/A
REVOKED	<i>(none)</i>	Terminal state — INV-003	N/A

⚠ Critical: Settlement Engine Exclusivity

Only the Settlement Engine may transition a Commitment out of the LOCKED state. No other service, adapter, or external caller has the authority to call `settle_commitment()` or `expire_commitment()` directly. Attempts to do so will be rejected with an `AuthorizationViolation` error.

2.3 Invariants

URIB enforces five mathematical and behavioral invariants throughout the system. These invariants are not soft conventions — they are asserted at runtime by dedicated invariant-check modules and tested by dedicated test harnesses in `urib/tests/invariants/`. A violation of any invariant is treated as a critical defect requiring immediate incident response.

INV-001

Conservation

The sum of all debits across a transaction graph must equal the sum of all credits. No value is created or destroyed within URIB. This invariant is enforced by

`MathPrimitives.assert_conservation(debits, credits)`, which raises an `INV001Violation` exception if the sums diverge. Conservation is checked at Commitment creation, at Bridge Swap initiation, and during batch sealing in the Audit Ledger.

INV-002

Idempotency

Re-submitting a Commitment with the same `commit_id` produces no additional side effects after the first acceptance. The CommitmentManager detects duplicate `commit_id` values at creation time and returns the existing Commitment record rather than creating a new one. This guarantee extends to all event emissions: duplicate events with the same `commit_id` are deduplicated by the Audit Ledger before appending. Idempotency keys for all operations are CSPRNG-derived 128-bit nonces where no stable domain key exists.

INV-003

Finality Monotonicity

A Commitment that has reached `SETTLED`, `EXPIRED`, or `REVOKED` state may never transition to any other state. These are terminal states. The CommitmentManager's state machine explicitly rejects any transition request on a terminal Commitment by raising `FinalityViolation`. This invariant is also enforced at the Commitment Store level with a database-layer constraint, providing defense in depth against any application-layer bypass.

INV-004

Rail Isolation

A Commitment may be bound to exactly one rail for its entire lifetime. The `rail_id` field is set at creation and is immutable. Cross-rail value transfer is not accomplished by re-binding a Commitment — it requires a **Bridge Swap** (see Section 5.3), which creates two separate Commitments, one on each rail, coordinated through an internal liquidity pool by the Bridge Swap Coordinator. Attempting to modify the `rail_id` of an existing Commitment raises `INV004Violation`.

INV-005

Denomination Integrity

The `amount` and `currency` fields of a Commitment are immutable after issuance. No rounding, conversion, normalization, or scaling occurs within a Commitment's lifecycle. If a consumer requires a different denomination, a Bridge Swap must be used to create a new Commitment in the target denomination. Any system component that attempts to mutate these fields on an existing Commitment

will receive an `INV005Violation` exception. This invariant, combined with fixed-point representation, ensures that no value is gained or lost through numeric representation artifacts.

2.4 Math Primitives

All monetary values in URIB are represented as **fixed-point integers** with an explicit scale factor. A fixed-point value consists of an integer mantissa and an integer scale, where the represented real value equals `amount / 10scale`. For example, `amount=10050, scale=2` represents exactly \$100.50. This representation is lossless, unambiguous, and immune to the rounding errors and precision drift that accompany IEEE 754 floating-point arithmetic.

⚠ Floating Point is Forbidden

The use of

`float`

,

`double`

, or any floating-point type to represent monetary values is strictly prohibited in URIB. Violations will be caught by automated linting rules in CI. Use only

`FixedPoint`

from

`urib.math`

.

Hash Functions: URIB uses two distinct hash algorithms for different purposes:

- **SHA-256** is used for `memo_hash` computation. This is a well-established standard that ensures interoperability with external audit and compliance systems that may independently verify memo hashes.
- **BLAKE3** is used for Merkle audit proofs and `finality_proof_hash` computation in SettlementReceipts. BLAKE3 is chosen for its superior throughput in high-volume batch operations and its resistance to length-extension attacks. It is used exclusively for internal cryptographic integrity rather than external interoperability.

Merkle Commitment Trees: Each settlement batch processed by the Audit Ledger is anchored to a Merkle root. Individual LifecycleEvents are hashed as leaves; sibling hashes are combined iteratively up the tree until a single root hash is produced. This root is published to an external notary service, allowing any party with a Merkle proof to verify that a specific event was included in a specific batch without needing access to the full Audit Ledger. The tree is built using

```
MathPrimitives.build_merkle_tree(leaves) and the root retrieved via  
MathPrimitives.get_merkle_root(tree).
```

Nonce Generation: All idempotency keys that cannot be derived from a stable domain identifier (such as a `commit_id`) are generated as CSPRNG-derived 128-bit nonces using the system's cryptographically secure pseudorandom number generator. This ensures that retry keys are statistically unique and unpredictable, preventing collision-based replay attacks.

3. Architecture Overview

3.1 High-Level Component Map

URIB is composed of eight distinct runtime components plus the Event Bus backbone. Each component has a strictly defined responsibility boundary; no component may perform operations outside its defined role.

Component	Role	Stateful?	Primary Communication
URIB Gateway	External-facing API surface. Validates inbound Commitment requests, enforces authentication and rate limits, routes validated requests to the Commitment Manager.	No	Synchronous RPC → CommitmentManager; inbound HTTP/gRPC
Commitment Manager (CM)	Owns the Commitment lifecycle state machine. Persists all Commitment records to the Commitment Store. Emits lifecycle events to the Event Bus on every state transition.	Yes (primary + replica)	Event Bus (emit); synchronous RPC from Gateway
Rail Dispatcher (RD)	Subscribes to LOCKED events on the Event Bus. Consults the RCM to select the appropriate Rail Adapter for the Commitment's <code>rail_id</code> . Dispatches settlement instructions to the selected adapter.	No	Event Bus (subscribe LOCKED); Event Bus (emit DISPATCHED)
Rail Adapters	One per rail (ACH, SWIFT, RTP, Stablecoin, Book Transfer). Each implements <code>IRailAdapter</code> . Translates URIB settlement instructions into rail-native protocol messages and returns structured SettlementReceipts.	No	Called by Rail Dispatcher; emit results to Event Bus
Settlement Engine (SE)	Receives settlement confirmation events from Rail Adapters via the Event Bus. Calls CommitmentManager to transition Commitments to SETTLED or EXPIRED. Emits SettlementReceipts to the Event Bus and Audit Ledger.	No	Event Bus (subscribe/emit); calls CM for state transitions
Audit Ledger	Append-only log of all Commitment lifecycle events and settlement proofs. Builds Merkle trees over event batches. Publishes batch roots to an external notary service. Used by compliance and dispute resolution.	Yes (append-only store)	Event Bus (subscribe all events); external notary (HTTP publish)
Bridge Swap Coordinator (BSC)	Orchestrates cross-rail swaps by chaining two Commitments through an internal liquidity pool. Manages swap atomicity: if either leg of the swap fails, the BSC initiates a compensating transaction.	No	Calls CM for Commitment creation; Event Bus for status monitoring

Component	Role	Stateful?	Primary Communication
Event Bus	Kafka-based pub/sub backbone. All inter-component communication (except the Gateway-to-CM synchronous call) flows through topic-partitioned Kafka streams. Provides at-least-once delivery with consumer-side idempotency (INV-002).	Yes (Kafka cluster)	Central; all services publish and subscribe

3.2 Data Flow — Commitment Lifecycle (Prose Description)

The following describes the end-to-end data flow for a standard Commitment from API submission to final settlement receipt. Each step references components by name.

1. **Submission:** A calling application submits a `CommitmentRequest` to the **URIB Gateway** via the external API (HTTP or gRPC). The Gateway validates the request structure, authenticates the caller, and applies rate-limit checks. If the request is valid, the Gateway issues a synchronous RPC call to the **Commitment Manager**.
2. **Creation:** The **Commitment Manager** receives the request, checks for duplicate `commit_id` (INV-002), validates field constraints, and creates a new Commitment record in `PENDING` state. It persists the record to the **Commitment Store** (PostgreSQL with primary/replica replication) and emits a `CREATED` LifecycleEvent to the **Event Bus**. The Commitment is returned synchronously to the Gateway, which relays it to the caller.
3. **Locking:** Upon receiving authorization to proceed (typically from the calling application invoking `lock_commitment()`), the **Commitment Manager** transitions the Commitment to `LOCKED` state, persists the update, and emits a `LOCKED` LifecycleEvent to the **Event Bus**.
4. **Dispatch:** The **Rail Dispatcher** is subscribed to the `LOCKED` topic on the **Event Bus**. Upon receiving the `LOCKED` event, it queries the **Rail Capability Matrix** to select the appropriate Rail Adapter for the Commitment's `rail_id`. It calls `IRailAdapter.submit_settlement(commitment)` on the selected adapter. A `DISPATCHED` event is emitted to the Event Bus.
5. **Rail Settlement:** The **Rail Adapter** translates the Commitment into a rail-native settlement instruction (e.g., an ACH entry, a SWIFT FIN message, an on-chain transaction) and submits it to the external settlement network. The adapter polls for confirmation (or receives an

asynchronous callback from the network). Once a confirmation is received, the adapter emits a settlement confirmation event to the **Event Bus** containing a raw confirmation payload.

6. **Finality Confirmation:** The **Settlement Engine** is subscribed to settlement confirmation events. It constructs a `SettlementReceipt` — computing the `finality_proof_hash` as the BLAKE3 hash of the raw confirmation payload — and calls `CommitmentManager.settle_commitment()` to transition the Commitment to `SETTLED` state. The `SETTLED` LifecycleEvent and the SettlementReceipt are emitted to the **Event Bus**.
7. **Audit Anchoring:** The **Audit Ledger** is subscribed to all LifecycleEvent topics. Every event — `CREATED`, `LOCKED`, `DISPATCHED`, `SETTLED` — is appended to the append-only event log in order. Every 60 seconds, the Audit Ledger seals the current batch, builds a Merkle tree over all events in the batch, and publishes the batch's Merkle root to the external notary service.
8. **Receipt Delivery:** The calling application, subscribed to the settlement confirmation topic for its `commit_id`, receives the `SettlementReceipt` and can proceed with product-layer ledger updates. The lifecycle is complete.

3.3 Deployment Topology

URIB components are deployed as independent, horizontally scalable services in a Kubernetes environment organized into logically isolated namespaces. The following topology principles govern all deployments:

- **Stateless Services:** The URIB Gateway, Rail Dispatcher, Settlement Engine, and Bridge Swap Coordinator are fully stateless and may be scaled horizontally without coordination. Their only shared state is mediated through the Event Bus.
- **Stateful Services:** The Commitment Manager relies on the Commitment Store (PostgreSQL), which runs in a primary/replica configuration. Only the CM primary may accept writes; replicas serve read queries for `get_commitment()` and `list_commitments()`. The Audit Ledger is backed by an append-only store that also runs in primary/replica mode.
- **Rail Adapter Isolation:** Each Rail Adapter is deployed in its own Kubernetes namespace with dedicated network policies, service accounts, and secret stores. This prevents a compromised or misbehaving adapter from affecting other rails or core components.
- **Communication Discipline:** All inter-service communication flows through the Event Bus (Kafka). The single exception is the synchronous validation path from the URIB Gateway to the Commitment Manager, which uses direct gRPC for latency-sensitive request validation and Commitment creation. No other direct RPC links exist between internal services.
- **Event Bus:** Kafka topics are partitioned by `commit_id` hash to ensure that all events for a given Commitment are processed by the same consumer partition in order, simplifying state reconstruction without requiring distributed locking.

Deployment Note

The

`make dev-up`

command (see Section 9) provisions a local single-node Kafka cluster, a local PostgreSQL instance, and containerized mock Rail Adapters that simulate rail behavior with configurable latency and failure injection. This environment is sufficient for all development and invariant-testing workflows.

4. Module Breakdown

The URIB codebase is organized as a monorepo under the root package `urib`. The table below summarizes all primary modules, their package paths, and their core responsibilities. Detailed per-module descriptions follow.

Module	Package Path	Responsibility
Gateway	<code>urib.gateway</code>	Ingress validation, auth, rate limiting
CommitmentManager	<code>urib.core.cm</code>	Lifecycle state machine, persistence
RailDispatcher	<code>urib.core.rd</code>	Rail selection, adapter routing
ACH Adapter	<code>urib.adapters.ach</code>	ACH rail integration
SWIFT Adapter	<code>urib.adapters.swift</code>	SWIFT rail integration
RTP Adapter	<code>urib.adapters.rtp</code>	Real-Time Payments rail integration
Stablecoin Adapter	<code>urib.adapters.stablecoin</code>	On-chain stablecoin settlement
Book Transfer Adapter	<code>urib.adapters.book</code>	Internal ledger book transfers
SettlementEngine	<code>urib.core.se</code>	Finality confirmation, state transitions
BridgeSwapCoordinator	<code>urib.core.bsc</code>	Cross-rail swap orchestration

Module	Package Path	Responsibility
AuditLedger	<code>urib.audit</code>	Append-only event log, Merkle anchoring
MathPrimitives	<code>urib.math</code>	Fixed-point arithmetic, hashing, Merkle
RailCapabilityMatrix	<code>urib.config.rcm</code>	Compile-time rail trait configuration

urib.gateway — Gateway

The Gateway is the sole external-facing surface of URIB. It handles TLS termination, caller authentication (mTLS + API key validation), JSON/Protobuf request deserialization, input validation (field presence, type checking, RCM-based rail existence verification), and per-caller rate limiting backed by a shared token-bucket store. The Gateway does not perform any business logic beyond validation; all decisions about Commitment creation and lifecycle management are delegated to the CommitmentManager via synchronous gRPC.

Design constraints: Must be fully stateless. Must validate all inputs before forwarding to CM — never pass malformed requests downstream. Rate-limit state is stored externally; the Gateway holds no in-process state beyond configuration.

urib.core.cm — CommitmentManager

The CommitmentManager is the authoritative owner of the Commitment lifecycle state machine. It is the only component permitted to create, update, or query Commitment records in the Commitment Store. Every state transition is validated against the permitted transition table, persisted atomically to PostgreSQL, and immediately followed by an Event Bus emission of the corresponding LifecycleEvent. The CM exposes a gRPC API for synchronous callers (the Gateway) and subscribes to select Event Bus topics for asynchronous coordination.

Design constraints: Stateful — requires primary/replica Commitment Store. All writes must be transactional with event emission (outbox pattern). Idempotent handlers only: duplicate requests with the same `commit_id` must return the existing record without side effects (INV-002). No floating-point arithmetic.

urib.core.rd — RailDispatcher

The RailDispatcher subscribes to the LOCKED topic on the Event Bus. For each LOCKED event, it retrieves the Commitment's `rail_id`, performs an RCM lookup to validate the rail is active and the Commitment's amount is within the rail's capacity limits, then routes the settlement instruction to the

appropriate Rail Adapter instance. The RailDispatcher does not interact directly with rails; it delegates entirely to the selected adapter. It emits a DISPATCHED event after successful delegation.

Design constraints: Must be stateless. Must never hard-code rail selection logic — use RCM exclusively. Must handle adapter unavailability gracefully with configurable retry and dead-letter queue fallback.

urib.adapters.ach — ACH Adapter

The ACH Adapter implements `IRailAdapter` for the Automated Clearing House network. It translates Commitment data into NACHA-formatted ACH entries, submits them to the ACH operator via the configured FTP/SFTP batch channel, and polls for return file processing. The adapter maps ACH trace numbers to URIB `commit_id` values for idempotency. It handles ACH return codes and maps them to URIB cancel or expiry events as appropriate.

Design constraints: Stateless service; all state is in the Commitment Store via CM. Idempotency enforced via ACH trace number registry. Must handle T+1 latency: the adapter may not receive settlement confirmation until the next business day.

urib.adapters.swift — SWIFT Adapter

The SWIFT Adapter implements `IRailAdapter` for SWIFT FIN and ISO 20022 message exchange. It constructs MT103 or pacs.008 messages from Commitment data, submits them through the SWIFT Alliance Gateway, and tracks settlement status using the Unique End-to-End Transaction Reference (UETR) as the idempotency key. It supports multi-currency Commitments and maps SWIFT acknowledgement, confirmation, and rejection messages to URIB lifecycle transitions.

Design constraints: No capacity cap on the rail, but adapter-level per-transaction monitoring is required for AML/sanctions screening integration. Settlement latency is variable (same-day to T+2); the adapter must handle long polling windows without holding open connections.

urib.adapters.rtp — RTP Adapter

The RTP Adapter implements `IRailAdapter` for the Real-Time Payments (RTP) network operated by The Clearing House. It submits Credit Transfer messages and expects near-instant positive or negative acknowledgement (within 10 seconds). The adapter enforces the \$1,000,000 per-transaction cap at submission time by consulting the RCM before dispatch. RTP transactions are irrevocable once acknowledged positively.

Design constraints: Must enforce the per-transaction cap before submission — do not rely solely on rail-side rejection. Idempotency enforced via RTP payment ID. Settlement is final and irreversible; no cancel path exists once a positive ACK is received.

urib.adapters.stablecoin — Stablecoin Adapter

The Stablecoin Adapter implements `IRailAdapter` for EVM-compatible on-chain stablecoin transfers (initially USDC on Ethereum mainnet). It constructs and signs ERC-20 transfer transactions, submits them to the configured RPC node, and monitors for transaction confirmation at a configurable block-depth finality threshold. The transaction hash serves as the idempotency key and is included in the `SettlementReceipt` as the `rail_ref`.

Design constraints: Gas price management must use a configurable strategy (e.g., EIP-1559 base fee + priority fee). Finality threshold must be set per-environment (e.g., 12 blocks on mainnet). Must handle RPC node failure with automatic failover to secondary node. No floating-point for wei or token amount calculations.

urib.adapters.book — Book Transfer Adapter

The Book Transfer Adapter implements `IRailAdapter` for instantaneous internal ledger transfers. It posts double-entry accounting records to the internal general ledger system, receives immediate confirmation, and generates a `SettlementReceipt`. Book transfers are always reversible and have no external dependency, making them the lowest-latency and highest-reliability rail available within URIB.

Design constraints: Settlement is synchronous and instant; the adapter must complete within the Rail Dispatcher's synchronous call timeout. Book transfers are the only rail where the adapter call is effectively synchronous end-to-end. All ledger entries must maintain double-entry balance (INV-001).

urib.core.se — SettlementEngine

The `SettlementEngine` is the exclusive authority for transitioning Commitments from `LOCKED` to `SETTLED` or `EXPIRED`. It subscribes to Rail Adapter confirmation events, validates the confirmation payload's integrity, constructs the `SettlementReceipt` (including computing the `finality_proof_hash` via BLAKE3), and calls `CommitmentManager` to apply the terminal state transition. It also monitors `expiry_epoch` values for all `LOCKED` Commitments and initiates expiry for those that time out without rail confirmation.

Design constraints: Must be stateless. The exclusive authority constraint for LOCKED→terminal transitions is enforced both by application-level authorization checks in the CM and by service-account-level network policies. Must be idempotent: receiving a duplicate confirmation for an already-SETTLED Commitment must produce no additional side effects.

urib.core.bsc — BridgeSwapCoordinator

The BridgeSwapCoordinator orchestrates cross-rail value transfers by creating two coordinated Commitments — one on the source rail (debit leg) and one on the target rail (credit leg) — through an internal liquidity pool. It manages the atomicity of the swap: if the debit leg settles but the credit leg fails, the BSC initiates a compensating book-transfer reversal. Swap state is tracked via a `SwapHandle`, which is a separate entity from the underlying Commitments.

Design constraints: Must be stateless. Swap atomicity is achieved through compensating transactions, not distributed two-phase commit. Each swap involves exactly two Commitments: one per rail (INV-004). Conservation across the swap (INV-001) is verified before the swap is initiated.

urib.audit — AuditLedger

The AuditLedger provides an append-only, cryptographically verifiable record of every LifecycleEvent emitted by the system. It subscribes to all event topics on the Event Bus, timestamps and signs each event, and appends it to the event store in order. Every 60 seconds it seals a batch: builds a Merkle tree over the batch's events, computes the root using BLAKE3, and publishes the root to the external notary service. It exposes APIs for Merkle proof generation, proof verification, and batch export.

Design constraints: The event store is strictly append-only — no update or delete operations are permitted at any layer. Callers must never write directly to the AuditLedger; all events arrive exclusively via Event Bus subscription. Batch sealing must be atomic: either all events in a batch are sealed together, or none are.

urib.math — MathPrimitives

MathPrimitives provides the complete set of mathematical building blocks used throughout URIB. It includes fixed-point arithmetic operations (add, subtract, scalar multiply) with scale-mismatch detection, conservation assertion, SHA-256 and BLAKE3 hashing, and Merkle tree construction and root extraction. This module is a zero-dependency library — it must not import any other URIB module.

Design constraints: No floating point anywhere in this module. All arithmetic operations are performed on Python `int` or equivalent arbitrary-precision integer types. All functions must be pure (no side effects, no I/O). Fully covered by property-based tests.

urib.config.rcm — RailCapabilityMatrix

The RailCapabilityMatrix is a compile-time configuration module that defines and exposes the trait profile of every registered rail. It is loaded at service startup and validated for completeness and internal consistency. All components that need to query rail capabilities must import and use the RCM — never access rail trait data from any other source. Adding a new rail requires creating a new `RailCapabilities` entry in this module and registering the corresponding `RailID`.

Design constraints: Must be fully validated at startup — any missing or inconsistent entry is a fatal startup error. Must be treated as read-only at runtime; no dynamic mutation of rail trait data is permitted after service initialization.

5. Function Interfaces

All interfaces below are expressed in Python-like pseudocode with type annotations. Concrete implementations in each module must conform exactly to these signatures. Parameter names, return types, and exception contracts are normative.

5.1 IRailAdapter Interface

All Rail Adapters must implement the `IRailAdapter` interface. No adapter may expose additional public methods that bypass this interface; all interaction between the Rail Dispatcher and an adapter must flow through these five methods.

```
interface IRailAdapter:
    def get_rail_id() -> RailID
    """
    Returns the RailID that uniquely identifies this adapter's rail.
    Must match the RailID registered in the RCM exactly.
    No parameters. Never raises.
    """
    def get_capabilities() -> RailCapabilities
    """
    Returns the RailCapabilities struct for this rail, as defined in
    the RCM.
    Used by the Rail Dispatcher to confirm capabilities before
    dispatch.
    No parameters. Never raises.
    """
    def submit_settlement(commitment: Commitment) -> SettlementReceipt
    """
    Translates the given Commitment into a rail-native settlement instruction
    and submits it to the external settlement network.
    Parameters:
    commitment (Commitment): The LOCKED Commitment to settle. Must be in
    LOCKED state; raises InvalidStateError otherwise.
    Returns:
    SettlementReceipt: Populated receipt with rail_ref and finality_proof_hash.
    For async rails (ACH, SWIFT), this may be a provisional
    receipt; the SE will await a final confirmation event.
    Raises:
    InvalidStateError -- Commitment is not in LOCKED state.
    RailCapacityError -- Commitment amount exceeds rail max capacity.
    RailUnavailableError -- External rail network is unreachable.
    DenominationMismatch -- Commitment currency not supported by this rail.
    """
    def poll_status(rail_ref: RailRef) -> SettlementStatus
    """
    Polls the external rail for the current settlement status of a previously
    submitted transaction identified by rail_ref.
    Parameters:
    rail_ref (RailRef): The rail-native reference returned by submit_settlement.
    Returns:
    SettlementStatus: One of PENDING, CONFIRMED, REJECTED,
    RETURNED.
    Raises:
    RailRefNotFound -- rail_ref is
    unknown to the external rail.
    RailUnavailableError -- External
    rail network is unreachable.
    """
    def cancel(rail_ref: RailRef,
    reason: CancelReason) -> CancelReceipt
    """
    Requests
    cancellation or recall of a previously submitted settlement.
    Only
    valid for reversible rails (e.g., ACH within return window, Book Transfer).
    Parameters:
    rail_ref (RailRef): The rail-native reference
    to cancel.
    reason (CancelReason): Structured reason for the
    cancellation.
    Returns:
    CancelReceipt: Confirmation of
    cancellation acceptance by the rail.
    Raises:
    IrreversibleRailError -- Rail does not support cancellation (e.g., RTP,
    SWIFT).
    CancelWindowExpired -- Reversal window has passed (ACH
    only).
    RailRefNotFound -- rail_ref is unknown to the
    external rail.
    """
```

5.2 CommitmentManager API

The CommitmentManager exposes the following API, accessible via gRPC from the Gateway and via internal service calls from the Settlement Engine. All methods are idempotent with respect to

`commit_id`.

```
def create_commitment(request: CommitmentRequest) -> Commitment """
Creates a new Commitment in PENDING state from the given request. Checks
for duplicate commit_id (INV-002) before creation. Parameters:
request (CommitmentRequest): Validated request containing rail_id, amount,
scale, currency, expiry_epoch, sender_handle,
receiver_handle, and memo string (will be hashed). Returns:
Commitment: The newly created (or existing, if duplicate) Commitment record
in PENDING state. Raises: InvalidRailError -- rail_id not
found in RCM. ValidationError -- Required fields missing or
malformed. ConservationError -- INV-001 violated in the containing
transaction graph. """
def lock_commitment(commit_id: UUID) ->
Commitment """ Transitions a PENDING Commitment to LOCKED state.
Parameters: commit_id (UUID): The UUID of the Commitment to lock.
Returns: Commitment: The updated Commitment in LOCKED state.
Raises: CommitmentNotFound -- No Commitment with this commit_id
exists. InvalidStateError -- Commitment is not in PENDING state.
FinalityViolation -- Commitment is in a terminal state (INV-003). """
def settle_commitment(commit_id: UUID, receipt: SettlementReceipt) ->
Commitment """ Transitions a LOCKED Commitment to SETTLED state.
Called exclusively by the Settlement Engine. Unauthorized callers receive
AuthorizationViolation. Parameters: commit_id (UUID):
The UUID of the Commitment to settle. receipt (SettlementReceipt):
The SettlementReceipt produced by the SE. Returns: Commitment:
The updated Commitment in SETTLED state with receipt attached. Raises:
AuthorizationViolation -- Caller is not the Settlement Engine.
CommitmentNotFound -- No Commitment with this commit_id exists.
InvalidStateError -- Commitment is not in LOCKED state.
FinalityViolation -- Commitment is already in a terminal state (INV-
003). """
def expire_commitment(commit_id: UUID) -> Commitment """
Transitions a PENDING or LOCKED Commitment to EXPIRED state. Called by
the Settlement Engine on timeout, or by the CM's expiry scheduler.
Parameters: commit_id (UUID): The UUID of the Commitment to expire.
Returns: Commitment: The updated Commitment in EXPIRED state.
Raises: AuthorizationViolation -- For LOCKED commitments, caller must
be Settlement Engine. CommitmentNotFound -- No Commitment with
this commit_id exists. FinalityViolation -- Commitment is
already in a terminal state (INV-003). """
def
revoke_commitment(commit_id: UUID, reason: RevokeReason) -> Commitment
""" Transitions a PENDING Commitment to REVOKED state. Only authorized
callers (compliance and admin service accounts) may invoke this method.
Parameters: commit_id (UUID): The UUID of the Commitment to
revoke. reason (RevokeReason): Structured reason for revocation.
Returns: Commitment: The updated Commitment in REVOKED state.
Raises: AuthorizationViolation -- Caller lacks revocation authority.
CommitmentNotFound -- No Commitment with this commit_id exists.
InvalidStateError -- Commitment is not in PENDING state; LOCKED
commitments cannot be revoked (use cancel
on the rail adapter first). FinalityViolation -- Commitment is
already in a terminal state (INV-003). """
def get_commitment(commit_id:
UUID) -> Commitment """ Retrieves the current state of a Commitment
```

```

by ID.      Parameters:      commit_id (UUID): The UUID of the Commitment
to retrieve.      Returns:      Commitment: The current Commitment record.
Raises:      CommitmentNotFound -- No Commitment with this commit_id
exists.      """ def list_commitments(filter: CommitmentFilter) ->
List[Commitment]      """ Returns a paginated list of Commitments matching
the given filter criteria.      Parameters:      filter
(CommitmentFilter): Filter struct supporting fields: rail_id, state,
sender_handle, receiver_handle, created_after_epoch,
created_before_epoch, page_size, page_token.      Returns:
List[Commitment]: Matching Commitment records in creation-time descending
order.      Raises:      ValidationError -- Filter fields are malformed or
contradictory.      """

```

5.3 BridgeSwapCoordinator API

The Bridge Swap Coordinator API enables atomic cross-rail value transfers. All three methods operate on a `SwapHandle`, which is distinct from and not interchangeable with a `commit_id`.

```

def initiate_swap(      source_commitment: Commitment,      target_rail_id:
RailID,      target_denomination: Currency ) -> SwapHandle      """
Initiates a cross-rail Bridge Swap. Creates two Commitments internally:      a
debit Commitment on source_commitment's rail and a credit Commitment      on
target_rail_id. Verifies INV-001 conservation across both legs before
locking either Commitment.      Parameters:      source_commitment
(Commitment): The fully formed, PENDING source Commitment
specifying the source rail, amount, and currency.      target_rail_id
(RailID):      The rail on which the credit leg will settle.
target_denomination (Currency):      The currency for the credit leg. If
different      from source, an FX rate
from the internal      liquidity pool
is applied.      Returns:      SwapHandle: An opaque handle identifying
the swap for status queries and aborts.      Raises:
ConservationError      -- INV-001 would be violated by this swap.
InvalidRailError      -- target_rail_id not found in RCM.
INV004Violation      -- Source Commitment already bound to a rail.
LiquidityUnavailable      -- Internal liquidity pool cannot fund the credit leg.
DenominationMismatch      -- target_denomination not supported by
target_rail_id.      """ def get_swap_status(swap_handle: SwapHandle) ->
SwapStatus      """ Returns the current status of a Bridge Swap.
Parameters:      swap_handle (SwapHandle): The SwapHandle returned by
initiate_swap.      Returns:      SwapStatus: One of PENDING,
DEBIT_LOCKED, DEBIT_SETTLED, CREDIT_LOCKED,      COMPLETED,
ABORTING, ABORTED.      Raises:      SwapNotFound -- No swap identified by
this SwapHandle exists.      """ def abort_swap(swap_handle: SwapHandle,
reason: AbortReason) -> AbortReceipt      """ Initiates abort of an in-
progress Bridge Swap. If the debit leg has already      settled, a
compensating book-transfer reversal is initiated automatically.      Aborts
are only possible before the credit leg reaches SETTLED state.
Parameters:      swap_handle (SwapHandle): The SwapHandle of the swap to
abort.      reason (AbortReason): Structured reason for the abort.
Returns:      AbortReceipt: Confirmation of abort initiation including
compensating      transaction commit_id if applicable.
Raises:      SwapNotFound      -- No swap identified by this SwapHandle
exists.      SwapAlreadyComplete -- Credit leg has already reached SETTLED
state; abort impossible.      """

```

5.4 AuditLedger API

The AuditLedger API is consumed by compliance engineers, dispute resolution workflows, and internal monitoring tooling. Direct writes to the AuditLedger from application code are not permitted; all

`append_event` calls are made exclusively by the AuditLedger's own Event Bus consumer.

```
def append_event(event: LifecycleEvent) -> EventRef """ Appends a
signed LifecycleEvent to the append-only event store. Called only by the
AuditLedger's internal Event Bus consumer – never by external callers.
Exposed here for documentation completeness. Parameters: event
(LifecycleEvent): The event to append. Must include timestamp,
emitting service signature, and commit_id. Returns: EventRef: An
opaque reference to the appended event (includes batch_id
and sequence number within the batch). Raises:
DuplicateEventError -- Event with identical (commit_id, event_type, epoch)
already exists (idempotency guard). SignatureError -- Event
signature verification fails. """ def get_merkle_proof(commit_id: UUID)
-> MerkleProof """ Retrieves a Merkle proof demonstrating that the
SETTLED event for the given Commitment is included in a sealed batch.
Parameters: commit_id (UUID): The UUID of the settled Commitment.
Returns: MerkleProof: Contains root_hash, leaf_hash, path, and
indices. Raises: CommitmentNotFound -- No events for this
commit_id exist. BatchNotSealed -- The batch containing this
event has not yet been sealed; try again after
the current seal interval (60 seconds). """ def verify_proof(proof:
MerkleProof) -> bool """ Verifies a MerkleProof by recomputing the
Merkle root from the leaf and path, then comparing to the published root
on the external notary. Parameters: proof (MerkleProof): The
proof to verify. Returns: bool: True if the proof is valid and
the root matches the notary record; False if the proof is
invalid or the root does not match. Raises: NotaryUnavailable --
External notary service is unreachable; verification
falls back to local root comparison with a warning. """ def
export_batch(batch_id: BatchID) -> AuditBatch """ Exports the
complete sealed batch identified by batch_id, including all
LifecycleEvents, SettlementReceipts, and the batch Merkle root.
sender_handle and receiver_handle are pseudonymized in the export unless
the caller holds an authorized compliance service account. Parameters:
batch_id (BatchID): The identifier of the sealed batch to export.
Returns: AuditBatch: Complete batch record with events, proofs, and
Merkle root. Raises: BatchNotFound -- No batch with this
batch_id exists. BatchNotSealed -- Batch is still open; export
not yet available. AuthorizationError -- Caller lacks export
authority. """
```

5.5 MathPrimitives

All functions in `urib.math` are pure functions with no side effects. They must be used as the exclusive mathematical toolkit for all monetary and cryptographic operations in the system.

```
def fixed_add(a: FixedPoint, b: FixedPoint) -> FixedPoint """ Adds
two FixedPoint values. Both must have identical scale factors.
Parameters:      a (FixedPoint): First operand.      b (FixedPoint):
Second operand. Returns:      FixedPoint: Sum with the same scale as
inputs. Raises:      ScaleMismatch -- a.scale != b.scale. """
def fixed_sub(a: FixedPoint, b: FixedPoint) -> FixedPoint """
Subtracts b from a. Both must have identical scale factors. Result must
be non-negative (monetary values may not be negative in URIB).
Parameters:      a (FixedPoint): Minuend.      b (FixedPoint):
Subtrahend. Returns:      FixedPoint: Difference with the same scale
as inputs. Raises:      ScaleMismatch -- a.scale != b.scale.
NegativeResult -- a.amount < b.amount (result would be negative). """
def fixed_mul_scalar(a: FixedPoint, scalar: int) -> FixedPoint """
Multiplies a FixedPoint value by an integer scalar. Used for fee
calculations and proportional splits. Parameters:      a
(FixedPoint): The fixed-point value to multiply.      scalar (int):
Integer multiplier. Must be non-negative. Returns:      FixedPoint:
Product with the same scale as input. Raises:      NegativeScalar --
scalar < 0. """
def assert_conservation(debits: List[FixedPoint],
credits: List[FixedPoint]) -> None """ Asserts that the sum of all
debits equals the sum of all credits (INV-001). All FixedPoint values in
both lists must share the same scale. Parameters:      debits
(List[FixedPoint]): List of debit amounts.      credits
(List[FixedPoint]): List of credit amounts. Returns:      None -- if
conservation holds. Raises:      ScaleMismatch -- Inconsistent
scale factors across inputs. INV001Violation -- Sum of debits != sum
of credits. """
def sha256_memo(memo: str) -> bytes """ Computes
the SHA-256 hash of the given memo string (UTF-8 encoded). Used
exclusively for populating Commitment.memo_hash. Parameters:
memo (str): The plaintext memo string. Returns:      bytes: 32-byte
SHA-256 digest. """
def blake3_hash(data: bytes) -> bytes """
Computes the BLAKE3 hash of the given byte string. Used for
finality_proof_hash computation and Merkle node hashing. Parameters:
data (bytes): Arbitrary byte input. Returns:      bytes: 32-byte
BLAKE3 digest. """
def build_merkle_tree(leaves: List[bytes]) ->
MerkleTree """ Constructs a binary Merkle tree from the given list of
leaf hashes. Leaves are sorted lexicographically before tree construction
to ensure deterministic root computation regardless of input ordering.
Parameters:      leaves (List[bytes]): List of 32-byte leaf hashes (e.g.,
BLAKE3 event hashes). Returns:      MerkleTree: Internal tree
structure from which root and proofs can be derived. Raises:
EmptyLeafSet -- leaves is empty; cannot build a tree with no leaves. """
def get_merkle_root(tree: MerkleTree) -> bytes """ Extracts the
Merkle root from a constructed MerkleTree. Parameters:      tree
(MerkleTree): A tree constructed by build_merkle_tree. Returns:
bytes: 32-byte BLAKE3 Merkle root hash. """
```

6. Rail Mappings

The following table is the human-readable representation of the Rail Capability Matrix. The authoritative, machine-readable definition lives in `urib.config.rcm`. This table is provided for reference and onboarding; do not treat it as normative in code — always consult the RCM programmatically.

Rail	Rail ID	Denomination	Finality Latency	Max Amount	Idempotency Key	Reversible	Adapter Module
ACH	<code>RAIL_ACH</code>	USD	T+1 business day	\$25,000,000	Trace number	Yes (within return window)	<code>urib.adapters.ach</code>
SWIFT	<code>RAIL_SWIFT</code>	Multi-currency	T+0 to T+2	No cap	UETR	No	<code>urib.adapters.swift</code>
RTP	<code>RAIL_RTP</code>	USD	<10 seconds	\$1,000,000	Payment ID	No	<code>urib.adapters.rtp</code>
Stablecoin (USDC)	<code>RAIL_SC_USDC</code>	USDC	~12 seconds (Ethereum)	No cap	Transaction hash	No	<code>urib.adapters.stablecoin</code>
Book Transfer	<code>RAIL_BOOK</code>	Internal units	Instant	No cap	Ledger entry ID	Yes (always)	<code>urib.adapters.book</code>

6.1 Conventional Rails

The five conventional rails above cover traditional financial networks. They form the baseline set registered in URIB at launch and each has deterministic finality detection and well-understood reversal windows.

6.2 Blockchain and DeFi Rails

URIB treats blockchain networks and Layer-2 protocols as first-class rails. Each blockchain rail is encapsulated behind its own Rail Adapter and registered in the RCM with on-chain-specific traits: block confirmation thresholds, gas/fee models, and smart contract interaction patterns. Developers must never call RPC nodes directly — all on-chain interaction flows through the respective Rail Adapter.

Rail	Rail ID	Denomination	Finality Latency	Confirmation Threshold	Max Amount	Idempotency Key	Reversible	Adapter Module
Ethereum L1	RAIL_ETH_L1	ETH / ERC-20	~6 min (12 blocks)	12 blocks	No cap	tx_hash	No	urib.adapters.eth_l1
Arbitrum (L2)	RAIL_ETH_ARB	ETH / ERC-20	~1 min	1 L2 block + L1 anchor	No cap	tx_hash + l2_batch_id	No	urib.adapters.eth_arb
Optimism (L2)	RAIL_ETH_OP	ETH / ERC-20	~2 min	1 L2 block + L1 anchor	No cap	tx_hash + l2_batch_id	No	urib.adapters.eth_op
Base (Coinbase L2)	RAIL_BASE	ETH / USDC	~2 min	1 L2 block + L1 anchor	No cap	tx_hash	No	urib.adapters.base
Polygon PoS	RAIL_POLYGON	MATIC / USDC	~5 min (256 blocks)	256 blocks	No cap	tx_hash	No	urib.adapters.polygon
Solana	RAIL_SOLANA	SOL / SPL tokens	~400 ms	32 slots (confirmed)	No cap	tx_signature	No	urib.adapters.solana
Bitcoin Lightning	RAIL_BTC_LN	BTC (satoshis)	Less than 5 seconds	Payment preimage reveal	0.04 BTC	payment_hash	No — HTLCs expire	urib.adapters.btc_ln
Avalanche C-Chain	RAIL_AVAX	AVAX / USDC.e	~2 seconds	1 block instant finality	No cap	tx_hash	No	urib.adapters.avax

Rail	Rail ID	Denomination	Finality Latency	Confirmation Threshold	Max Amount	Idempotency Key	Reversible	Adapter Module
TRON	RAIL_TRON	TRX / USDT-TRC20	~3 seconds (19 blocks)	19 blocks	No cap	tx_id	No	urib.adapters.tron
Stellar	RAIL_STELLAR	XLM / USDC	~5 seconds	Ledger close	No cap	tx_hash	No	urib.adapters.stellar

6.3 Crypto Rail Design Notes

Block Confirmation Thresholds

Finality on probabilistic chains (Ethereum L1, Polygon) is not immediate. The RCM stores a `min_confirmations` trait per rail. The Settlement Engine will not emit SETTLED until the Rail Adapter reports the required confirmation depth. Never override confirmation thresholds in application code.

Gas and Fee Abstraction

All network fees — gas, priority fees, Solana compute units, Lightning routing fees — are paid from a URIB-managed fee reserve. Fee costs are never deducted from the Commitment amount. INV-005 (Denomination Integrity) prohibits it. The Fee Reserve Manager (FRM) is a separate internal service not covered in this guide.

Smart Contract Rail Adapters

Stablecoin and DeFi rails (ETH L1, Arbitrum, Optimism, Base, Polygon, Avalanche) use smart contract dispatch. Each adapter holds a reference to a deployed URIB Settlement Contract on-chain. The Settlement Contract emits a `SettlementFinalized(commit_id, amount, receiver)` event that the adapter monitors via a log subscription. The `finality_proof_hash` in the `SettlementReceipt` is the BLAKE3 hash of the raw ABI-encoded event log.

HTLC Model for Lightning

Bitcoin Lightning settlements use Hash Time-Locked Contracts (HTLCs). The `Commitment expiry_epoch` maps directly to the HTLC timeout. If the HTLC is not claimed before expiry, the Commitment transitions to EXPIRED — INV-003 applies. The `payment_hash` serves as the idempotency key.

Multi-Token Denomination Handling

Rails like `RAIL_ETH_L1` support multiple ERC-20 token denominations. The `currency` field in the Commitment must specify the token contract address in addition to the ticker symbol, for example: `USDC@0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48`. The adapter resolves the contract and validates the denomination at dispatch time.

RCM Lookup Pattern

At runtime, the Rail Dispatcher selects the appropriate adapter for a given Commitment using the following pattern:

9. The Commitment's `rail_id` is extracted from the LOCKED event payload.
10. `RailCapabilityMatrix.get(rail_id)` is called. If the `rail_id` is not registered, the dispatcher raises `InvalidRailError` and dead-letters the event for manual review.
11. The returned `RailCapabilities` struct is inspected: the Commitment's `amount` is checked against `capabilities.max_amount`, and the Commitment's `currency` is checked against `capabilities.denominations`.
12. If all checks pass, the dispatcher instantiates (or retrieves from its adapter registry) the Rail Adapter class registered for the given `rail_id` and calls `adapter.submit_settlement(commitment)`.

Adapter Registration

New adapters are registered in the Rail Dispatcher's adapter registry map in

`urib.core.rd.registry`

. This map associates each

`RailID`

string with the corresponding

`IRailAdapter`

implementation class. Both the RCM entry and the registry entry must be added when introducing a new rail; the service will fail its startup health check if a RCM-registered rail has no corresponding adapter in the registry.

7. Settlement Outputs

7.1 SettlementReceipt

The `SettlementReceipt` is the primary artifact produced upon successful settlement of a Commitment. It is generated by the Settlement Engine, attached to the SETTLED Commitment record, and emitted to the Event Bus and the Audit Ledger. It is the definitive proof that a settlement occurred on a specific rail at a specific time.

Field	Type	Description
<code>receipt_id</code>	UUID	Globally unique identifier for this receipt.
<code>commit_id</code>	UUID	The <code>commit_id</code> of the Commitment that was settled.
<code>rail_id</code>	RailID	The rail on which settlement occurred. Must match <code>Commitment.rail_id</code> .
<code>rail_ref</code>	RailRef	The rail-native settlement reference (e.g., ACH trace number, SWIFT UETR, Ethereum tx hash).
<code>settled_at_epoch</code>	int (Unix epoch)	The Unix timestamp at which finality was confirmed by the rail.
<code>amount</code>	FixedPoint	The settled amount. Must match <code>Commitment.amount</code> and <code>Commitment.scale</code> (INV-005).
<code>currency</code>	Currency	The settled currency. Must match <code>Commitment.currency</code> (INV-005).
<code>finality_proof_hash</code>	bytes (32)	BLAKE3 hash of the concatenated raw rail confirmation payload. See below.
<code>adapter_version</code>	str	Semantic version string of the Rail Adapter that produced this receipt. Used for auditability of adapter behavior over time.

The `finality_proof_hash` is computed as `blake3_hash(raw_confirmation_payload)`, where `raw_confirmation_payload` is the complete, unmodified bytes of the settlement confirmation message received from the external rail (e.g., the full ACH return record, the SWIFT FIN acknowledgement, or the serialized Ethereum transaction receipt). This hash is stored in both the `SettlementReceipt` and the Audit Ledger, enabling dispute resolution teams to independently verify that the hash matches the actual rail confirmation by recomputing it from the raw message.

7.2 MerkleProof

A `MerkleProof` is a cryptographic inclusion proof that demonstrates a specific `LifecycleEvent` (typically the `SETTLED` event for a given `commit_id`) was included in a specific sealed `AuditBatch`. It is the primary artifact used in dispute resolution to prove that a settlement event was recorded and anchored at a specific point in time.

Field	Type	Description
<code>root_hash</code>	bytes (32)	The Merkle root of the sealed batch. This value is also published to the external notary service.
<code>leaf_hash</code>	bytes (32)	The BLAKE3 hash of the specific <code>LifecycleEvent</code> being proven.
<code>path</code>	List[bytes]	Ordered list of sibling node hashes along the path from leaf to root.
<code>indices</code>	List[int]	Corresponding list of left/right position indicators for each sibling in the path (0 = left sibling, 1 = right sibling).

Verification Algorithm: To verify a `MerkleProof`, begin with `current_hash = leaf_hash`. For each step `i` in the path: if `indices[i] == 0` (sibling is on the left), compute `current_hash = blake3_hash(path[i] + current_hash)`; if `indices[i] == 1` (sibling is on the right), compute `current_hash = blake3_hash(current_hash + path[i])`. After processing all path elements, compare `current_hash` to `root_hash`. If they match, and `root_hash` matches the published root on the external notary for the relevant `batch_id`, the proof is valid and the event's inclusion is cryptographically confirmed.

7.3 AuditBatch

An `AuditBatch` is the unit of export from the AuditLedger. Batches are sealed every 60 seconds. A sealed batch is immutable: no events can be added, removed, or modified after sealing.

Field	Type	Description
<code>batch_id</code>	BatchID	Globally unique identifier for this batch.
<code>batch_epoch</code>	int (Unix epoch)	The Unix timestamp at which this batch was sealed.
<code>commitment_count</code>	int	The number of distinct <code>commit_id</code> values represented by events in this batch.
<code>merkle_root</code>	bytes (32)	The BLAKE3 Merkle root computed over all events in this batch. Published to the external notary.
<code>events</code>	List[LifecycleEvent]	The complete ordered list of LifecycleEvents included in this batch.
<code>export_timestamp</code>	int (Unix epoch)	The Unix timestamp at which this export was generated (distinct from <code>batch_epoch</code>).

Batch Anchoring: Every 60 seconds, the AuditLedger's batch sealing process atomically collects all events received since the previous seal, computes their BLAKE3 hashes, builds a Merkle tree, extracts the root, and publishes it to the external notary service via a signed HTTP POST. The notary records the root hash, the `batch_id`, and the timestamp, providing a tamper-evident external anchor. This means that even if the URIB Audit Ledger itself were compromised, an auditor could verify the integrity of historical batches by comparing reconstructed roots against the notary's published records. Batches are sealed on a strictly time-based schedule — there is no minimum event count required to trigger sealing. An empty batch (no events in the 60-second window) will still be sealed with an empty Merkle tree, producing a deterministic root for that time slot.

8. Audit Flows

8.1 Real-Time Event Streaming

Every transition in the Commitment lifecycle produces a `LifecycleEvent` that is published to the Event Bus and consumed by the AuditLedger's dedicated subscriber. The six event types that the AuditLedger records are:

Event Type	Emitting Component	Trigger
CREATED	CommitmentManager	New Commitment successfully persisted in PENDING state.
LOCKED	CommitmentManager	Commitment transitioned from PENDING to LOCKED.
DISPATCHED	RailDispatcher	Settlement instruction delivered to Rail Adapter.
SETTLED	Settlement Engine	Rail confirmed settlement; Commitment in SETTLED state.
EXPIRED	Settlement Engine / CommitmentManager	Commitment reached expiry_epoch without settlement confirmation.
REVOKED	CommitmentManager	Authorized actor revoked a PENDING Commitment.

Each `LifecycleEvent` carries: the `commit_id`, the event type, a nanosecond-precision timestamp, the identity of the emitting service, and a digital signature produced with the emitting service's private key. The AuditLedger verifies this signature before appending the event. Events are guaranteed to arrive at the AuditLedger in monotonically increasing sequence numbers per `commit_id`, enforced by Kafka topic partition assignment on `commit_id` hash.

8.2 Batch Sealing & Merkle Anchoring

The AuditLedger operates a background batch-sealing process on a strict 60-second interval. The sealing sequence proceeds as follows:

13. At the seal interval boundary, the AuditLedger takes an atomic snapshot of all events received since the previous seal.
14. Each event's canonical byte representation is hashed with BLAKE3 to produce a leaf hash.
15. `MathPrimitives.build_merkle_tree(leaves)` is called to construct the Merkle tree.
16. `MathPrimitives.get_merkle_root(tree)` extracts the root hash.
17. The root hash, `batch_id`, and `batch_epoch` are signed by the AuditLedger's service key and published to the external notary via signed HTTP POST.
18. The batch is marked as sealed; its events are frozen and available for proof generation and export.

⚠ Developer Note: Event Bus vs. Direct Query

Developers must not query the AuditLedger for individual events during an active (unsealed) batch window. The AuditLedger does not expose a real-time event stream for unsealed batches. For live event monitoring (e.g., in development or observability tooling), subscribe directly to the relevant Event Bus topics. The AuditLedger API is appropriate only for sealed, historical batches.

8.3 Dispute Resolution Flow

The following step-by-step procedure governs all settlement disputes raised by counterparties or internal teams:

19. **Claim Submission:** The claimant (internal team or external counterparty via compliance liaison) submits a dispute record containing the `commit_id` in question and a description of the discrepancy.
20. **Record Retrieval:** The dispute resolution system retrieves the Commitment via `CommitmentManager.get_commitment(commit_id)` and its associated `SettlementReceipt` from the Commitment Store.
21. **Proof Retrieval:** `AuditLedger.get_merkle_proof(commit_id)` is called to obtain the Merkle proof for the SETTLED event associated with the Commitment.
22. **Proof Verification:** `AuditLedger.verify_proof(proof)` is called. If this returns `False`, or if a `NotaryUnavailable` exception is raised, the dispute is immediately escalated to the URIB Engineering on-call team for manual review.
23. **Batch Export:** If proof verification succeeds, `AuditLedger.export_batch(batch_id)` is called for the batch containing the SETTLED event, providing the full audit record for the dispute window.
24. **Hash Comparison:** The `SettlementReceipt.finality_proof_hash` is compared to the independently recomputed BLAKE3 hash of the raw confirmation payload retrieved from the Rail Adapter's confirmation archive. These must match.
25. **Resolution:** If all verifications pass and hashes match, the dispute is marked **RESOLVED**. If any verification fails or hashes mismatch, the dispute is escalated to **ESCALATED** status and routed to the rail-specific integration team and compliance for manual investigation.

8.4 Compliance Exports

Compliance teams are authorized to call `AuditLedger.export_batch(batch_id)` for any historical batch using a designated compliance service account. Exports include all LifecycleEvents, associated MerkleProofs, and SettlementReceipts for the batch window, presented in a structured format compatible with regulatory reporting requirements.

GDPR Pseudonymization: In all compliance exports, the `sender_handle` and `receiver_handle` fields within LifecycleEvents are automatically pseudonymized using a reversible tokenization scheme. The tokenization mapping is held by a separately access-controlled token vault. Only compliance officers holding an authorized token-vault read credential may retrieve the original identity behind a pseudonymous handle. All access to the token vault is itself logged to an independent audit log outside the URIB system perimeter.

9. Developer Quickstart

This section provides a step-by-step guide for getting a local URIB development environment operational from scratch. All commands assume a Unix-compatible shell environment (Linux or macOS; WSL2 recommended on Windows).

Step 1 — Environment Setup

Clone the URIB monorepo and bring up the local development stack:

```
# Clone the monorepo git clone git@internal.urib.eng:urib/urib-monorepo.git
cd urib-monorepo # Install prerequisites (Docker, docker-compose, Python
3.11+) # Consult docs/PREREQUISITES.md for platform-specific instructions #
Spin up local Event Bus (Kafka), Commitment Store (PostgreSQL), # and all
mock Rail Adapters make dev-up # Verify all services are healthy make
health-check
```

After `make dev-up`, the following local services are available: Kafka on `localhost:9092`, PostgreSQL on `localhost:5432`, URIB Gateway on `localhost:8080`, and mock Rail Adapters on their configured ports. Mock adapters simulate rail latency and can be configured to inject failures via environment variables in `config/local/mock-adapters.env`.

Step 2 — Running Tests

The full test suite — including unit tests, integration tests, and invariant harnesses — is executed with:

```
# Run the full test suite make test # Run only invariant harnesses (INV-001
through INV-005) make test-invariants # Run tests for a specific module make
test MODULE=urib.core.cm # Run with verbose output and coverage report make
test-coverage
```

All five invariant harnesses are located in `urib/tests/invariants/` and named

`test_inv_001_conservation.py` through `test_inv_005_denomination_integrity.py`.

These tests use property-based testing via Hypothesis to generate adversarial inputs. New engineers are strongly encouraged to read through these test files as a practical introduction to how each invariant is enforced.

Step 3 — Implementing a New Rail Adapter

26. Create a new module at `urib/adapters/<rail_name>/adapter.py`.
27. Implement the `IRailAdapter` interface (all five methods: `get_rail_id`, `get_capabilities`, `submit_settlement`, `poll_status`, `cancel`).
28. Register the new `RailID` and its `RailCapabilities` struct in `urib/config/rcm.py`.
29. Register the adapter class in the Rail Dispatcher's adapter registry in `urib/core/rd/registry.py`.
30. Create a mock adapter implementation in `urib/adapters/<rail_name>/mock.py` for local development and testing.
31. Add the adapter's namespace and deployment configuration to `deploy/k8s/adapters/<rail_name>/`.
32. Run `make test MODULE=urib.adapters.<rail_name>` and ensure all invariant harnesses pass with the new adapter registered.

Step 4 — Submitting a Commitment (Local)

Use the `urib-cli` command-line tool to submit a test Commitment against the local development environment:

```
# Submit a Book Transfer Commitment for $100.50 urib-cli commit create \
--rail RAIL_BOOK \          --amount 10050 \          --scale 2 \          --currency USD \
--receiver test-receiver \  --memo "Test payment for onboarding" # The
CLI prints the commit_id and initial PENDING state # Output: {"commit_id":
"...", "state": "PENDING", ...} # Lock the Commitment to trigger settlement
dispatch urib-cli commit lock --commit-id <commit_id> # Tail real-time
lifecycle events for this Commitment urib-cli events tail --commit-id
<commit_id>
```

For Book Transfer rail, settlement is instant. You should see `PENDING → LOCKED → SETTLED` in the event stream within milliseconds of locking. For mock ACH or SWIFT adapters, configure the simulated latency in `config/local/mock-adapters.env` to observe asynchronous settlement behavior.

Step 5 — Verifying an Audit Proof

```
# Wait for the current batch to seal (up to 60 seconds after settlement) #
Then verify the Merkle proof for the settled Commitment urib-cli audit verify
--commit-id <commit_id> # Expected output on success: # Fetching Merkle
proof for commit_id <commit_id>... # Proof retrieved: batch_id=<batch_id>,
leaf_hash=<hex> # Verifying proof against notary root... # ✓ Proof VALID.
Settlement of commit_id <commit_id> is cryptographically confirmed. # Export
the full audit batch for inspection urib-cli audit export-batch --batch-id
<batch_id> --output ./audit_export.json
```

Step 6 — Common Pitfalls

⚠ Critical Pitfalls for New Engineers

- **Never use float for monetary values.** All amounts must use `FixedPoint` from `urib.math`. Cl linting will reject `float` and `Decimal` types in value paths.
- **Never hard-code rail behavior.** Do not branch on rail IDs, hard-code capacity limits, or assume latency characteristics in any module outside of `urib.config.rcm` and Rail Adapter implementations.
- **Never transition Commitment state outside the CommitmentManager.** No other service, adapter, or tool may write to the Commitment Store or emit state-transition events directly.
- **Never write directly to the AuditLedger.** The Audit Ledger is event-bus-driven only. Attempts to call `append_event()` from application code will be rejected by service-account authorization policies.
- **Never skip the RCM lookup.** Even if you "know" a rail's characteristics from documentation, always use the RCM programmatically. Documentation may lag behind RCM updates.

10. Glossary

The following terms have precise, system-specific meanings within URIB. Use these definitions consistently in code comments, pull request descriptions, and internal communications.

Rail

Any settlement network — including ACH, SWIFT, RTP, stablecoin ledgers, or internal book-transfer systems — abstracted behind the uniform `IRailAdapter` interface. From URIB's perspective, all rails are interchangeable at the interface level; differences are encapsulated within Rail Adapters.

Commitment

A cryptographically bound, time-gated, immutable record representing a promise to settle a specific value transfer on a specific rail. The central primitive of URIB. Commitments progress through a strict lifecycle: `PENDING` → `LOCKED` → `SETTLED` | `EXPIRED` | `REVOKED`.

`commit_id`

A universally unique identifier (UUID v4) assigned to a Commitment at creation time. Also functions as the primary idempotency key for the Commitment — re-submitting a request with the same `commit_id` returns the existing Commitment without creating a duplicate (INV-002).

Rail Capability Matrix (RCM)

A compile-time configuration table in `urib.config.rcm` that maps every registered `RailID` to its complete set of Rail Traits (denomination, finality latency, capacity limits, idempotency mechanism, reversibility). The single source of truth for all rail behavior; must always be consulted programmatically rather than through hard-coded assumptions.

Rail Adapter

A service module that implements the `IRailAdapter` interface for a specific settlement rail. Responsible for translating URIB Commitment data into rail-native protocol messages, submitting them to the external network, and returning structured `SettlementReceipt` objects. Each adapter is deployed in its own isolated Kubernetes namespace.

Settlement Engine (SE)

The URIB component with exclusive authority to transition Commitments from `LOCKED` to `SETTLED` or `EXPIRED`. Receives settlement confirmation events from Rail Adapters, constructs `SettlementReceipt` objects, and calls the `CommitmentManager` to apply terminal state transitions.

SettlementReceipt

The primary artifact produced upon successful settlement of a Commitment. Contains the rail-native reference (`rail_ref`), settlement timestamp (`settled_at_epoch`), settled amount and currency, and a `finality_proof_hash` computed as the BLAKE3 hash of the raw rail confirmation payload. Stored in the Commitment record and the Audit Ledger.

MerkleProof

A cryptographic inclusion proof demonstrating that a specific LifecycleEvent is contained within a sealed AuditBatch. Consists of a root hash, leaf hash, sibling path, and position indices. Used in dispute resolution to verify that a settlement event was recorded and anchored at a specific time.

AuditBatch

A sealed, immutable collection of LifecycleEvents covering a 60-second window. Each batch has a Merkle root published to an external notary service. Batches are the unit of compliance export and the anchor point for Merkle proofs.

LifecycleEvent

A signed, timestamped record of a single Commitment state transition, emitted to the Event Bus and consumed by the AuditLedger. One of: `CREATED`, `LOCKED`, `DISPATCHED`, `SETTLED`, `EXPIRED`, `REVOKED`.

Event Bus

The Kafka-based pub/sub backbone that mediates all inter-component communication within URIB (except the synchronous Gateway-to-CommitmentManager gRPC path). All LifecycleEvents, settlement confirmations, and coordination messages flow through the Event Bus. Topics are partitioned by `commit_id` hash for ordered, per-Commitment event processing.

Bridge Swap

A cross-rail value transfer orchestrated by the Bridge Swap Coordinator. Implemented as two coordinated Commitments — one debit leg on the source rail and one credit leg on the target rail — settled through an internal liquidity pool. Bridge Swaps maintain INV-001 (conservation) across both legs and handle failure compensation via reversing book-transfer transactions.

FixedPoint

URIB's internal representation of monetary values. A pair of integers `(amount: int, scale: int)` where the represented real value equals `amount / 10scale`. Floating-point types are prohibited throughout the system. All arithmetic on monetary values must use `MathPrimitives` functions.

finality_proof_hash

A 32-byte BLAKE3 hash of the raw, unmodified confirmation payload received from an external settlement rail. Stored in the `SettlementReceipt` and the Audit Ledger, enabling independent verification that the receipt accurately reflects the rail's actual confirmation message.

CSPRNG

Cryptographically Secure Pseudorandom Number Generator. Used in URIB exclusively for generating 128-bit nonces for idempotency keys in cases where no stable domain key exists (e.g., internal operation IDs that cannot be derived from a `commit_id`). Standard library CSPRNG (e.g., `os.urandom`) is mandated; `random` module is prohibited.

INV (Invariant)

A mathematically or behaviorally enforced guarantee that must hold at all times within the URIB system. There are five invariants: INV-001 (Conservation), INV-002 (Idempotency), INV-003 (Finality Monotonicity), INV-004 (Rail Isolation), and INV-005 (Denomination Integrity). Violations are treated as critical defects requiring immediate incident response.

11. Implementing Additional Rail Adapters

This section guides engineers through the process of adding a new financial rail to URIB. Follow the prerequisites, checklist, and patterns below to ensure your adapter is production-ready and fully interoperable with the commitment lifecycle.

11.1 When to Add a New Rail

A new rail adapter is warranted when all three of the following prerequisites are satisfied:

33. The target rail has a stable, documented API or protocol with predictable error semantics that can be mapped to the URIB `IRailAdapter` interface.
34. The rail denomination can be expressed as a fixed-precision decimal unit supported by URIB `MathPrimitives` (or a new primitive can be registered without violating existing invariants).
35. A responsible engineering team or team member is assigned as the long-term adapter owner, who will maintain health checks, monitor rail operator status pages, and respond to URIB-RD-003 alerts.

11.2 Implementation Checklist

Complete all 11 steps before merging a new adapter to the main branch:

36. Register a new `RAIL_XXX` constant in `urib.rail.registry.RailRegistry` with `denomination`, `max_amount`, `finality_latency_seconds`, and `reversible` flag.
37. Create the adapter package at `urib.adapters.your_rail_name` following the package naming convention.
38. Implement the `IRailAdapter` interface: `submit()`, `cancel()`, `status()`, and `health_check()` methods.
39. Implement idempotency: the `submit()` method must be safe to call multiple times with the same `commit_id` and must return the same result on duplicate calls.
40. Map all rail-specific error codes to URIB-RA error codes in a dedicated `error_mapper.py` module within the adapter package.

41. Implement the `health_check()` method to verify connectivity to the rail within a configurable timeout (default 5 seconds).
42. Configure circuit-breaker parameters: `failure_threshold` (default 5), `recovery_timeout_seconds` (default 30) in the adapter helm chart values.
43. Write unit tests covering all five mock scenarios: `SUCCESS`, `TIMEOUT`, `DOUBLE_SPEND`, `RATE_LIMITED`, and `REORG` (blockchain only).
44. Write an integration test against the rail sandbox or testnet environment that exercises the full commitment lifecycle: `CREATED` → `LOCKED` → `DISPATCHED` → `SETTLED`.
45. Add the new `RAIL_XXX` to the Rail Capability Matrix in the developer documentation (Section 6).
46. Submit a runbook update for RB-005 listing the new rail adapter's restart command, health check endpoint, and escalation contact.

11.3 Mock Adapter Pattern

Use the `MockRailAdapter` class during local development and unit testing. Configure it to return deterministic outcomes by setting the `scenario` parameter:

Scenario	Trigger Value	Adapter Behaviour	Maps To
SUCCESS	<code>scenario='SUCCESS'</code>	Returns SETTLED immediately	Normal happy path
TIMEOUT	<code>scenario='TIMEOUT'</code>	Sleeps beyond deadline and raises	URIB-RA-001
DOUBLE_SPEND	<code>scenario='DOUBLE_SPEND'</code>	Returns duplicate detection error	URIB-RA-003
RATE_LIMITED	<code>scenario='RATE_LIMITED'</code>	Returns capacity-exceeded error	URIB-RD-002
REORG	<code>scenario='REORG'</code>	Returns block reorg signal	URIB-RA-015

```
from urib.adapters.mock import MockRailAdapter adapter =  
MockRailAdapter(scenario='TIMEOUT', delay_seconds=10) result =  
adapter.submit(commitment) # raises URIBError(code='URIB-RA-001')
```

11.4 Blockchain Adapter Extra Requirements

Adapters targeting on-chain rails must also satisfy the following five requirements beyond the base `IRailAdapter` interface:

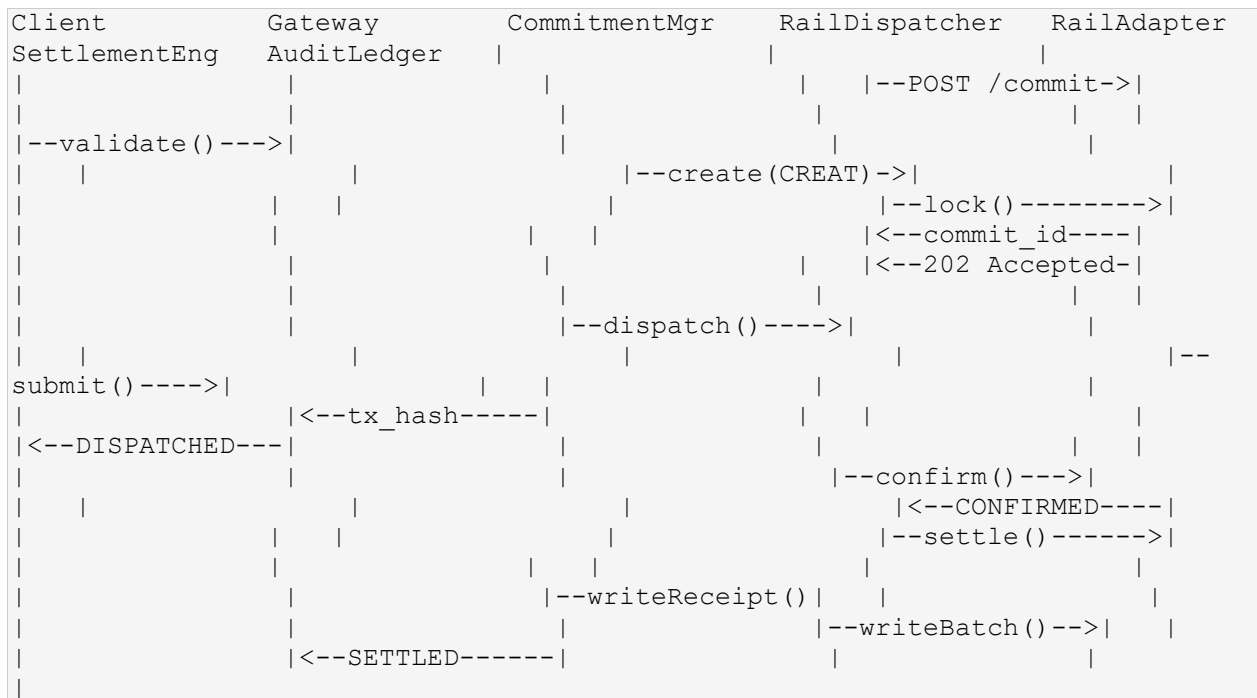
- 47. RPC Failover:** Maintain a list of at least two RPC endpoint URLs and implement automatic failover when the primary returns HTTP 5xx or times out. Log each failover event as `WARN`.
- 48. Confirmation Threshold Configuration:** Expose a `confirmations_required` config parameter (e.g., 12 for Ethereum L1, 64 for Polygon). Never hard-code confirmation counts.
- 49. Gas / Fee Abstraction:** The adapter must handle gas estimation internally. Callers pass only the commitment amount; the adapter computes and applies the fee using the configured gas strategy (e.g., EIP-1559 base+tip on EVM chains).
- 50. Reorg Detection:** Subscribe to `newHeads` events (or equivalent) and compare the block hash of previously confirmed transactions against the canonical chain on each new head. Emit URIB-RA-015 if a discrepancy is detected.
- 51. Smart Contract Address Pinning:** All contract addresses used by the adapter must be pinned in the helm chart `ConfigMap` and verified at startup against the on-chain bytecode hash stored in the adapter config. Startup fails loudly if hashes do not match.

12. Sequence Diagrams

The following diagrams illustrate the major flows through the URIB system. Each step is numbered and followed by a prose explanation. ASCII diagrams are provided for environments where rendered diagrams are unavailable.

12.1 Standard Commitment Lifecycle — Happy Path

Shows a single commitment moving from creation through settlement across one rail.



52. Client submits a commitment via POST /commit with rail_id, amount, denomination, and expiry.

53. Gateway validates schema and authentication; rejects immediately on URIB-GW-001/-002/-004/-005.

54. CommitmentManager creates the record in CREATED state and transitions to LOCKED, reserving the amount.

55. Gateway returns HTTP 202 Accepted with commit_id to the client.

56. RailDispatcher selects the registered adapter for the given rail_id.

57. RailAdapter.submit() sends the transaction to the upstream rail and returns a tx_hash.

12.3 Dispute Resolution Flow

Shows how a disputed commitment is investigated and resolved.

Operator	Gateway	CommitmentMgr	RailAdapter	AuditLedger
SettlementEng				
		--GET /commit-		
			--inspect()---->	
			<--state:LOCKED-	
		--query audit-		
<--getBatch()--			<--batch_proof--	
		--verify_proof		
			--status()----->	
				<--
railStatus()-				
--CONFIRMED----->				--force_settle()-----
----->				
--settle()----->				<--resolved---

69. Operator queries commitment state and finds it stuck in LOCKED.

70. Operator retrieves the AuditBatch containing the commitment and verifies the Merkle proof.

71. Operator queries the rail adapter directly to confirm the on-rail status.

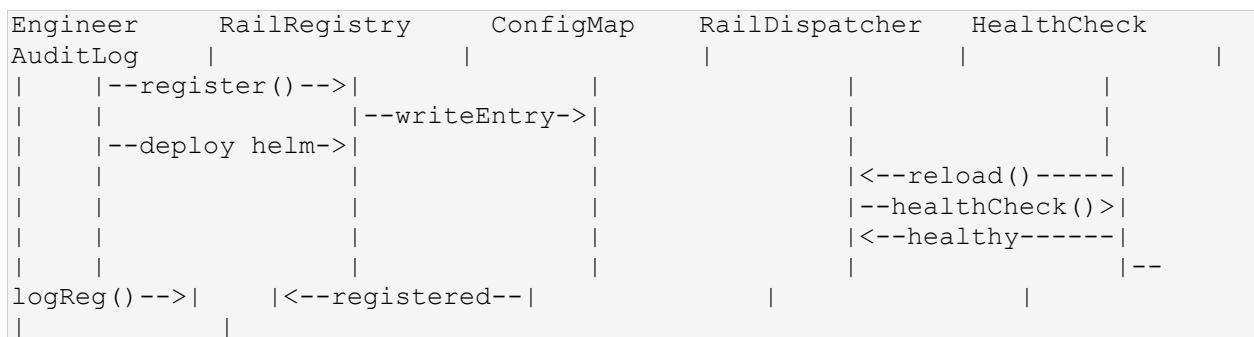
72. Rail adapter reports CONFIRMED — the commitment has settled on rail but not in URIB.

73. Operator issues force_settle() with the rail tx_hash as evidence.

74. CommitmentManager transitions to SETTLED. AuditLedger records the forced settlement with operator trace_id.

12.4 New Rail Adapter Registration Flow

Shows the steps for registering a newly implemented adapter into a live URIB deployment.



75. Engineer calls RailRegistry.register() with the new RAIL_XXX entry and adapter class reference.

76. RailRegistry writes the entry to the ConfigMap.

77. Engineer deploys the adapter via helm upgrade.

78. RailDispatcher reloads its registry on config change signal.

79. RailDispatcher issues an immediate health_check() to the new adapter.

80. If healthy, the adapter is marked ACTIVE and the registration event is written to the AuditLog.

81. If the health check fails, the adapter is registered but marked INACTIVE; URIB-RD-003 is raised and the adapter will not receive traffic until it passes a health check.

13. Error Code Reference

All errors emitted by URIB components share a common envelope. Errors are machine-readable and designed for structured log aggregation and alerting pipelines.

13.1 Error Envelope Schema

Every URIB error is serialised as a `URIBError` dataclass:

```
@dataclass class URIBError:      code: str                                # e.g.
"URIB-CM-002"      message: str                                # Human-readable
description      severity: Literal["WARN", "ERROR", "CRITICAL"]
retry_safe: bool                                # True = caller may retry immediately
component: str                                # Emitting component name      commit_id:
Optional[str]      # Relevant commitment ID, if any      rail_id:
Optional[str]      # Rail involved, if any      timestamp_epoch: int
# Unix epoch milliseconds      trace_id: str                                #
Distributed trace correlation ID
```

CRITICAL severity errors must page on-call immediately and halt further processing on the affected commitment. Retry-safe errors may be retried with exponential back-off capped at 32 seconds.

13.2 Gateway Errors — URIB-GW

Code	Severity	retry_safe	Trigger Condition	Notes
URIB-GW-001	ERROR	Yes	Malformed or unparseable request payload	Validate schema before forwarding
URIB-GW-002	ERROR	No	API key missing or signature invalid	Rotate credentials; do not retry
URIB-GW-003	WARN	Yes	Client rate limit exceeded	Honour Retry-After header
URIB-GW-004	ERROR	No	rail_id not registered in RailRegistry	Check rail_id spelling and registration
URIB-GW-005	ERROR	No	Denomination not supported by requested rail	Verify denomination before calling

13.3 Commitment Manager Errors — URIB-CM

Code	Severity	retry_safe	Trigger Condition	Notes
URIB-CM-001	ERROR	No	commit_id already exists in ledger	Use idempotency: re-submit same commit_id
URIB-CM-002	ERROR	No	State machine transition rejected (e.g. SETTLED → LOCKED)	Inspect current state before retrying
URIB-CM-003	ERROR	No	Commitment with given commit_id not found	Verify commit_id; may have expired
URIB-CM-004	ERROR	No	Commitment already in SETTLED state	Operation is a no-op; treat as success
URIB-CM-005	CRITICAL	No	INV-001 conservation invariant violated	Halt processing; page on-call immediately
URIB-CM-006	ERROR	No	expiry_epoch is in the past at submission time	Re-create commitment with valid expiry

13.4 Rail Dispatcher Errors — URIB-RD

Code	Severity	retry_safe	Trigger Condition	Notes
URIB-RD-001	ERROR	No	No adapter registered for the given rail_id	Register adapter or correct rail_id
URIB-RD-002	WARN	Yes	Rail capacity window currently full	Retry after back-off; consider alternative rail
URIB-RD-003	WARN	Yes	Adapter health-check returned unhealthy	Dispatcher will route around; retry in 5 s

13.5 Rail Adapter Errors — URIB-RA

Standard adapter errors apply to all rails. Blockchain-specific codes (RA-010+) apply only to on-chain rail adapters.

Standard Rail Errors

Code	Severity	retry_safe	Trigger Condition	Notes
URIB-RA-001	WARN	Yes	Adapter did not respond within deadline	Increase timeout or retry on next adapter instance
URIB-RA-002	ERROR	No	Rail explicitly rejected the transaction	Inspect rejection reason; do not retry without modification
URIB-RA-003	ERROR	No	Rail detected duplicate transaction	Commitment is already submitted; poll status instead
URIB-RA-004	WARN	Yes	Transaction is in intermediate confirming state	Poll with exponential back-off; do not cancel
URIB-RA-005	ERROR	No	Rail declined the cancel/reversal request	Manual intervention required; escalate to ops

Blockchain-Specific Errors

Code	Severity	retry_safe	Trigger Condition	Notes
URIB-RA-010	WARN	Yes	RPC node unreachable or returning 5xx	Failover to secondary RPC endpoint
URIB-RA-011	ERROR	No	On-chain smart contract execution reverted	Decode revert reason; fix parameters before retry
URIB-RA-012	WARN	Yes	Gas estimation call failed	Retry with manual gas limit; check node health
URIB-RA-013	WARN	Yes	Mempool congestion; transaction not propagated	Increase gas tip (EIP-1559) or wait and retry
URIB-RA-014	ERROR	No	Lightning HTLC preimage not revealed before CLTV expiry	Funds returned to sender; re-initiate payment
URIB-RA-015	ERROR	No	Block reorganisation detected; finality assumption violated	Suspend commitment; run RB-003 runbook

13.6 Settlement Engine Errors — URIB-SE

Code	Severity	retry_safe	Trigger Condition	Notes
URIB-SE-001	WARN	Yes	Merkle proof not yet available for commitment	Retry after proof generation completes
URIB-SE-002	CRITICAL	No	Double-settlement detected on same commit_id	Halt; trigger financial reconciliation immediately
URIB-SE-003	WARN	Yes	Audit ledger write failed during settlement	Retry write; settlement itself may have succeeded

13.7 Bridge Swap Coordinator Errors — URIB-BSC

Code	Severity	retry_safe	Trigger Condition	Notes
URIB-BSC-001	ERROR	Yes	Source leg failed before target leg initiated; no funds at risk	Safe to retry full swap
URIB-BSC-002	CRITICAL	No	Source settled but target leg failed; partial swap state	Do NOT initiate new swap; run RB-002 immediately
URIB-BSC-003	WARN	Yes	Insufficient liquidity on target rail at this time	Retry after liquidity refill or choose alternate rail
URIB-BSC-004	ERROR	No	Swap aborted by operator kill-switch	Await operator clearance before reattempting

13.8 Audit Ledger Errors — URIB-AUD

Code	Severity	retry_safe	Trigger Condition	Notes
URIB-AUD-001	WARN	Yes	Audit batch write failed; batch will be retried	No data loss; retry is automatic
URIB-AUD-002	ERROR	No	Merkle proof verification failed on sealed batch	Integrity breach; escalate to security team
URIB-AUD-003	ERROR	No	Attempted write to an already-sealed batch	Write to the next open batch

13.9 Invariant Violation Errors — URIB-INV



POLICY:

All INV-xxx errors carry CRITICAL severity and `retry_safe=False`. On receipt of any INV error, the receiving service **MUST**: (1) immediately halt further mutations on the affected commitment, (2) publish an incident event to the ops alerting bus, (3) await explicit operator clearance before resuming.

Code	Severity	retry_safe	Trigger Condition	Notes
URIB-INV-001	CRITICAL	No	Conservation invariant violated: $\text{sum}(\text{inputs}) \neq \text{sum}(\text{outputs})$	Halt; financial reconciliation required
URIB-INV-002	CRITICAL	No	State machine transitioned to illegal terminal state	Inspect state log; rollback if possible
URIB-INV-003	CRITICAL	No	Audit batch Merkle root mismatch vs ledger	Security escalation; do not seal further batches
URIB-INV-004	CRITICAL	No	Commitment expiry violated: settlement after <code>expiry_epoch</code>	Reverse settlement; open incident
URIB-INV-005	CRITICAL	No	Rail denomination invariant violated across bridge swap legs	Suspend swap coordinator; reconcile balances

14. Runbook Appendix

Each runbook below is self-contained. Work through steps in order. Confirm the alert source in the URIB monitoring dashboard before beginning.

RB-001: Commitment Stuck in LOCKED State

Field	Detail
Trigger	Commitment has been in LOCKED state for more than 5 minutes with no outbound rail dispatch observed.
Impact	Funds are reserved but not yet dispatched. No financial loss if caught within TTL. SLA clock is running.
SLA	Resolve within 30 minutes of alert.
Immediate Steps	1. Run <code>urib-cli commitment inspect --id [commit_id]</code> to confirm state and timestamps. 2. Check RailDispatcher logs for URIB-RD-001 or URIB-RD-003 errors in the commitment window. 3. If adapter unhealthy (RD-003), verify adapter process is running and restart if needed. 4. If rail_id unregistered (RD-001), escalate to rail engineering immediately. 5. If expiry is within 10 minutes, escalate to L2 now.
Root Cause Checklist	Rail adapter process crash or OOM kill; RailRegistry config rolled back without restarting dispatcher; Network partition between dispatcher and adapter; Upstream rail outage causing health checks to fail.
Resolution	Once adapter is healthy, dispatcher will auto-retry. Confirm commitment transitions to DISPATCHED within 60 seconds. If not, manually trigger: <code>urib-cli commitment retry --id [commit_id]</code>
Escalation	Page L2 On-Call if unresolved at 30 minutes. Open P1 incident immediately if <code>expiry_epoch</code> is breached.

RB-002: Partial Swap Failure (URIB-BSC-002)

⚠ WARNING:

Do NOT initiate a new swap or retry the swap coordinator for the affected swap_id

. A new swap will double-spend the source funds. Await explicit finance team clearance before any action.

Field	Detail
Trigger	URIB-BSC-002 CRITICAL alert. Source leg SETTLED on Rail A; target leg FAILED before reaching DISPATCHED on Rail B.
Impact	Level-1 incident. Funds have left the source rail but have not arrived on the target rail. Financial exposure is open.
SLA	4 hours to full resolution or escalation to CRO.
Immediate Steps	1. Acknowledge alert and open P1 incident ticket. 2. Run <code>urib-cli swap inspect --swap-id [swap_id]</code> to capture the state of both legs. 3. Record <code>source_commit_id</code> (SETTLED), <code>target_commit_id</code> (FAILED), amount, target rail, and timestamp. 4. Notify Finance Reconciliation team within 15 minutes. 5. DO NOT call any retry or re-initiate endpoints on the swap coordinator. 6. Check target rail adapter logs for specific URIB-RA-* failure code.
Root Cause Checklist	Unrecoverable adapter timeout (RA-001) after source settled; Smart contract revert on target chain (RA-011) mid-swap; Liquidity exhausted after source leg committed; Network partition isolating target adapter.
Resolution	Finance team manually reconciles open exposure and credits destination account out-of-band. Once reconciled, mark target commitment CANCELLED with audit note referencing the P1 incident ID. Close incident only after reconciliation is confirmed.
Escalation	Escalate to CRO and Head of Engineering if unresolved at 4 hours. Provide full P1 incident summary.

RB-003: Blockchain Block Reorg (URIB-RA-015)

Field	Detail
Trigger	URIB-RA-015 alert raised by a blockchain rail adapter. A confirmed transaction has been reorganised out of the canonical chain.
Impact	A commitment believed to be SETTLED may no longer be settled on-chain. Financial exposure depends on reorg depth and duration.
SLA	Assess reorg depth within 15 minutes. Resolve or escalate within 2 hours.
Immediate Steps	1. Identify <code>commit_id</code> and <code>tx_hash</code> from the RA-015 error payload. 2. Query the RPC node for current status of the original tx (<code>eth_getTransactionByHash</code> or equivalent). If null, the transaction has been dropped from the mempool. 3. Determine reorg depth by comparing original tx block number to current canonical head. 4. If depth is within the rail's confirmation threshold, the adapter will auto-reconfirm — monitor for CONFIRMED within 10 minutes. 5. If depth exceeds threshold, suspend the commitment and escalate immediately. 6. Do not resubmit until the chain has stabilised (minimum 3 canonical blocks).
Root Cause Checklist	Shallow reorg (1–2 blocks): normal chain operation, threshold is sufficient guard; Deep reorg (>6 blocks on Ethereum): potential 51% attack; RPC node connected to minority fork: switch RPC endpoint; Confirmation threshold set too low for current network conditions.
Resolution	Once the tx re-appears in the canonical chain and accumulates required confirmations, the adapter emits CONFIRMED and the commitment re-settles automatically. Verify via: <code>urib-cli commitment inspect --id [commit_id]</code>
Escalation	Reorg depth >6 blocks: page Security team and CTO. Suspected 51% attack: follow exchange security protocol immediately.

RB-004: Invariant Violation CRITICAL (URIB-INV)



CRO must be notified within 1 hour of any INV-001 or INV-003 alert, regardless of resolution status.

Field	Detail
Trigger	Any URIB-INV-001 through URIB-INV-005 CRITICAL PagerDuty alert fires.
Impact	A mathematical or integrity invariant of the system has been violated. Financial data integrity is at risk. All processing on the affected commitment halts automatically.
SLA	CRO notification within 1 hour. Full root cause within 24 hours. Post-mortem within 72 hours.
Immediate Steps	1. Activate URIB intake kill-switch: <code>urib-cli admin intake-pause --reason "INV violation [INV-00X]"</code> . 2. Page L2 engineer and Head of Engineering simultaneously. 3. Capture full state dump: <code>urib-cli commitment dump --id [commit_id] --include-ledger</code> . 4. Snapshot the AuditLedger batch containing the affected commitment before any further writes. 5. Notify CRO with: invariant code, <code>commit_id</code> , amount at risk, and current status. 6. Do NOT attempt manual reconciliation until root cause is established.
Root Cause Checklist	INV-001: input/output sum mismatch — check race conditions or FX rounding in CommitmentManager; INV-002: illegal state transition — inspect state machine for missing guard clauses in recent deploys; INV-003: Merkle root mismatch — check AuditLedger for concurrent writes; INV-004: settlement after expiry — check NTP sync and clock skew between services; INV-005: denomination mismatch — inspect BridgeSwapCoordinator normalisation logic.
Resolution	Requires signed-off root cause analysis and a code or config fix. Finance confirms reconciled balances before kill-switch is lifted. Re-enable only after Head of Engineering approval: <code>urib-cli admin intake-resume --approved-by [name]</code>
Escalation	CRO within 1 hour. CTO and Board Risk Committee if exposure exceeds configured threshold. External auditor notification per regulatory obligations for INV-001 or INV-003.

RB-005: Rail Adapter Outage

Field	Detail
Trigger	URIB-RD-003 alert fires continuously for a specific <code>rail_id</code> . Adapter circuit breaker has opened. Health checks consistently failing.
Impact	All new commitments targeting the affected rail are rejected. In-flight commitments queued or stuck in LOCKED state. No financial loss if TTLs are respected.
SLA	Restore adapter or activate failover rail within 60 minutes.
Immediate Steps	1. Check adapter circuit breaker status: <code>urib-cli rail status --rail-id [RAIL_ID]</code> . 2. Review adapter process logs for root cause (crash, OOM, upstream rail outage, config error). 3. If adapter process has crashed, restart it: <code>kubectl rollout restart deployment/urib-adapter-[rail_id] -n urib-prod</code> . 4. After restart, monitor health check: <code>urib-cli rail healthcheck --rail-id [RAIL_ID] --watch</code> . 5. If upstream rail is down, activate fallback if configured: <code>urib-cli rail failover --rail-id [RAIL_ID] --to [FALLBACK_RAIL_ID]</code> . 6. Communicate estimated recovery time to operations team via ops channel.

Field	Detail
Root Cause Checklist	Adapter container OOM killed by Kubernetes — increase memory limits in helm values; Upstream rail operator incident — check rail operator status page; TLS certificate expired on adapter-to-rail mTLS connection; ConfigMap update broke adapter startup — check recent ConfigMap changes; Rate limit imposed by upstream rail operator.
Resolution	Once health checks pass for 3 consecutive intervals, circuit breaker auto-closes and RailDispatcher resumes routing. Verify by watching a test commitment through to DISPATCHED. Drain the LOCKED queue: <code>urib-cli commitment retry-queue --rail-id [RAIL_ID]</code>
Escalation	If outage persists beyond 60 minutes with no failover available, page L2 on-call and notify affected downstream services. If rail is a primary settlement rail, escalate to CTO.